РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет искусственного интеллекта

Дисциплина: Инструменты обработки и визуализации данных

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2

Студент: Князев Арсений Михайлович

Группа: ЗФИбд-01-24

Вариант № 7

In [1]:
```
import os
import sys
from pathlib import Path

from pyspark import SparkConf, SparkContext

os.environ["PYSPARK_PYTHON"] = sys.executable
os.environ.setdefault("SPARK_LOCAL_IP", "127.0.0.1")

conf = (
    SparkConf()
    .setMaster("local[*]")
    .setAppName("SocialGraphLab2")
    .set("spark.ui.showConsoleProgress", "false")
)
sc = SparkContext.getOrCreate(conf=conf)
sc.setLogLevel("ERROR")

print("spark:", sc.version)
```

```
spark: 4.0.1
```

Шаг 1 - делаем граф симметричным.

Шаг 2 - собираем друзей каждого пользователя.

Шаг 3 - ищем друзей друзей и убираем уже существующие связи.

Тестовые данные

In [2]:
```
DATA_FILE = Path("social_edges.txt")

edges_text = """Анна,Борис
Анна,Вера
Борис,Глеб
Вера,Глеб
Вера,Дина
Глеб,Егор
Дина,Егор
Егор,Жанна
Дина,Илья
"""

DATA_FILE.write_text(edges_text, encoding="utf-8")
print(DATA_FILE.read_text(encoding="utf-8"))
```

```
Анна,Борис
Анна,Вера
Борис,Глеб
Вера,Глеб
Вера,Дина
Глеб,Егор
Дина,Егор
Егор,Жанна
Дина,Илья
```

Шаг 1

In [3]:
```
edges = sc.textFile(str(DATA_FILE)).map(lambda line:
tuple(line.strip().split(",")))

# каждую связь записываем в обе стороны
symmetric_edges = edges.flatMap(lambda edge: [edge, (edge[1],
edge[0])]).distinct().cache()

for user, friend in sorted(symmetric_edges.collect()):
    print(f"{user:6} -> {friend}")
```

```
Анна   -> Борис
Анна   -> Вера
Борис  -> Анна
Борис  -> Глеб
Вера   -> Анна
Вера   -> Глеб
Вера   -> Дина
Глеб   -> Борис
Глеб   -> Вера
Глеб   -> Егор
Дина   -> Вера
Дина   -> Егор
Дина   -> Илья
Егор   -> Глеб
Егор   -> Дина
Егор   -> Жанна
Жанна  -> Егор
Илья   -> Дина
```

Шаг 2

In [4]:
```
friends_by_user = (
    symmetric_edges.groupByKey().mapValues(lambda friends:
sorted(set(friends))).cache()
)

for user, friends in sorted(friends_by_user.collect()):
    print(f"{user:6}: {', '.join(friends)}")
```

```
Анна  : Борис, Вера
Борис : Анна, Глеб
Вера  : Анна, Глеб, Дина
Глеб  : Борис, Вера, Егор
Дина  : Вера, Егор, Илья
Егор  : Глеб, Дина, Жанна
Жанна : Егор
Илья  : Дина
```

Шаг 3

In [5]:
```
second_level = (
    friends_by_user.flatMapValues(lambda friends: friends)
    .map(lambda pair: (pair[1], pair[0]))
    .join(friends_by_user)
    .map(lambda item: (item[1][0], item[1][1]))
    .flatMapValues(lambda friends: friends)
    .distinct()
)

# убираем самого пользователя и его прямых друзей
self_pairs = friends_by_user.map(lambda item: (item[0], item[0]))
excluded = symmetric_edges.union(self_pairs).distinct()

friends_of_friends = (
    second_level.subtract(excluded).groupByKey().mapValues(lambda friends:
sorted(set(friends)))
)
```

Результат

In [6]:
```
for user, friends in sorted(friends_of_friends.collect()):
    print(f"{user:6}: {', '.join(friends)}")
```

```
Анна  : Глеб, Дина
Борис : Вера, Егор
Вера  : Борис, Егор, Илья
Глеб  : Анна, Дина, Жанна
Дина  : Анна, Глеб, Жанна
Егор  : Борис, Вера, Илья
Жанна : Глеб, Дина
Илья  : Вера, Егор
```